

Chapter 4

Control Statements,

Part 2

An introduction to the control structures in C

Learning Objectives

- Identify the essentials of counter-controlled iteration.
- Use C++20's text-formatting capabilities.
- Use the for and do...while iteration statements.
- Perform multiple selection using the switch selection statement.
- Use the [[fallthrough]] attribute in switch statements.
- Use selection statements with initializers.
- Use the break and continue statements to alter the flow of control.
- Use the logical operators to form compound conditions.
- Understand the challenges of processing monetary amounts.
- Understand why you should not use floating-point data to hold monetary values.

Introduction

- This chapter continues our presentation of structured-programming theory and principles by introducing all but one of C++'s remaining control statements—for, do...while, switch, break and continue.
- We explore the essentials of counter-controlled iteration.
- We use compound-interest calculations to begin investigating the issues of processing monetary amounts and the representational errors associated with floating-point types.

Introduction

- We use a switch statement to count the number of A, B, C, D and F grade equivalents in a set of numeric grades.
 - We show if and switch statements capabilities that allow you to initialize one or more variables in these statements' headers.
- We discuss the logical operators, which enable you to combine simple conditions to form compound conditions.

Introduction.

- We introduce C++20's powerful and expressive text-formatting capabilities, which are based on those in Python, Microsoft's .NET languages (like C# and Visual Basic) and Rust.
- We discuss the challenges of processing monetary amounts with floating-point types like double. Then, we represent and manipulate monetary amounts precisely using objects of a class from the Boost Multiprecision Library.

Essentials of Counter-Controlled Iteration

- A loop is a group of instructions the computer executes repeatedly while some **loop-continuation condition** remains true.
- We've discussed two means of iteration:
 - Counter-controlled iteration
 - Sentinel-controlled iteration
- **Counter-controlled iteration is sometimes called definite iteration** because we know in advance exactly how many times the loop will be executed.
- **Sentinel-controlled iteration is sometimes called indefinite iteration** because it's not known in advance how many times the loop will be executed.

Essentials of Counter-Controlled Iteration (Cont.)

- In counter-controlled iteration, a **control variable is used to count** the number of iterations.
- The control variable is incremented (usually by 1) each time the group of instructions is performed.
- When the value of the control variable indicates that the correct number of iterations has been performed, the loop terminates and execution continues with the statement after the iteration statement.

Essentials of Counter-Controlled Iteration (Cont.)

- Sentinel values are used to control iteration when:
 - The precise number of iterations isn't known in advance, and
 - The loop includes statements that obtain data each time the loop is performed.
- The **sentinel** value indicates “end of data.”
- The sentinel is entered after all regular data items have been supplied to the program.
- Sentinels **must be distinct from regular data items.**

Essentials of Counter-Controlled Iteration (Cont.)

- Counter-controlled iteration requires:
 - the **name** of a control variable (or loop counter).
 - the **initial value** of the control variable.
 - the loop-continuation condition that determines if looping should continue, and
 - the **increment** (or **decrement**) by which the control variable is modified each time through the loop.

Essentials of Counter-Controlled Iteration (Cont.)

- Lines 6, 8 and 10 define the elements of counter-controlled iteration.
 - Line 6 declares the control variable (counter) as an int, reserves space for it in memory and initializes it to 1.
 - ◆ Declarations that require initialization are executable statements.
 - ◆ Variable declarations that also reserve memory are definitions.

```
1 // Counter-controlled iteration with the while iteration statement.
2 #include <iostream>
3 using namespace std;
4
5 int main() {
6     int counter{1}; // declare and initialize control variable
7
8     while (counter <= 10) { // loop-continuation condition
9         cout << counter << " ";
10        ++counter; // increment control variable
11    }
12
13    cout << "\n";
14 }
```

1 2 3 4 5 6 7 8 9 10

Essentials of Counter-Controlled Iteration (Cont.)

- Lines 6, 8 and 10 define the elements of counter-controlled iteration.
 - The loop-continuation condition (line 8) tests whether the control variable's value is less than or equal to —the final value for which the condition is true.
 - ◆ The loop terminates when the control variable exceeds.

```
1 // Counter-controlled iteration with the while iteration statement.
2 #include <iostream>
3 using namespace std;
4
5 int main() {
6     int counter{1}; // declare and initialize control variable
7
8     while (counter <= 10) { // loop-continuation condition
9         cout << counter << " ";
10        ++counter; // increment control variable
11    }
12
13    cout << "\n";
14 }
```

1 2 3 4 5 6 7 8 9 10

Essentials of Counter-Controlled Iteration (Cont.)

- Lines 6, 8 and 10 define the elements of counter-controlled iteration.
 - Line 9 displays 's value once per loop iteration.
 - Line 10 increments the control variable by 1 for each loop iteration.

```
1 // Counter-controlled iteration with the while iteration statement.
2 #include <iostream>
3 using namespace std;
4
5 int main() {
6     int counter{1}; // declare and initialize control variable
7
8     while (counter <= 10) { // loop-continuation condition
9         cout << counter << " ";
10        ++counter; // increment control variable
11    }
12
13    cout << "\n";
14 }
```

1 2 3 4 5 6 7 8 9 10

Essentials of Counter-Controlled Iteration.

- Always control counting loops with integers.
 - Floating-point values are approximate.
 - Controlling counting loops with floating-point variables can result in imprecise counter values and inaccurate termination tests that prevent a loop from terminating.
- Placing a semicolon immediately after the right parenthesis of a while header makes that while's body an empty statement. This usually is a logic error.

for Iteration Statement

- The `for` iteration statement handles all the details of counter-controlled iteration
 - When lines 8–10 begin executing, the `for` statement declares control variable `counter` and initializes it to 1.
 - Next, it tests the loop-continuation condition between the two required semicolons (`counter <= 10`).

```
1 // Counter-controlled iteration with the for iteration statement.
2 #include <iostream>
3 using namespace std;
4
5 int main() {
6     // for statement header includes initialization,
7     // loop-continuation condition and increment
8     for (int counter{1}; counter <= 10; ++counter) {
9         cout << counter << " ";
10    }
11
12    cout << "\n";
13 }
```

for Iteration Statement (Cont.)

- The `for` iteration statement handles all the details of counter-controlled iteration
 - Because counter's initial value is 1, the condition is true.
 - ◆ So, line 9 displays counter's value (1).
 - After executing line 9, the loop executes `++counter` to increment counter by 1.

```
1 // Counter-controlled iteration with the for iteration statement.
2 #include <iostream>
3 using namespace std;
4
5 int main() {
6     // for statement header includes initialization,
7     // loop-continuation condition and increment
8     for (int counter{1}; counter <= 10; ++counter) {
9         cout << counter << " ";
10    }
11
12    cout << "\n";
13 }
```

for Iteration Statement (Cont.)

- The `for` iteration statement handles all the details of counter-controlled iteration
 - Then the program performs the loop-continuation test again to determine whether to proceed with the loop's next iteration.
 - ◆ At this point, counter's value is 2, and the condition is still true, so the program executes line 9 again.

```
1 // Counter-controlled iteration with the for iteration statement.
2 #include <iostream>
3 using namespace std;
4
5 int main() {
6     // for statement header includes initialization,
7     // loop-continuation condition and increment
8     for (int counter{1}; counter <= 10; ++counter) {
9         cout << counter << " ";
10    }
11
12    cout << "\n";
13 }
```


for Iteration Statement (Cont.)

- The `for` iteration statement handles all the details of counter-controlled iteration
 - This process continues until the loop has displayed the numbers 1–10, and counter's value becomes 11. At this point, the loop-continuation test fails, iteration terminates, and the program continues with the first statement after the loop (line 12).

```
1 // Counter-controlled iteration with the for iteration statement.
2 #include <iostream>
3 using namespace std;
4
5 int main() {
6     // for statement header includes initialization,
7     // loop-continuation condition and increment
8     for (int counter{1}; counter <= 10; ++counter) {
9         cout << counter << " ";
10    }
11
12    cout << "\n";
13 }
```

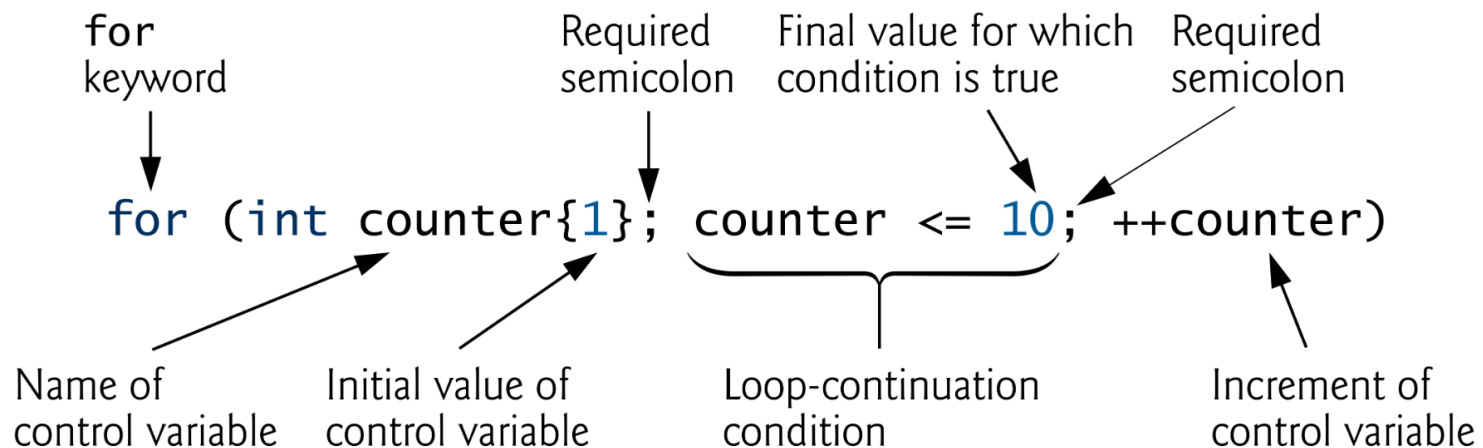
for Iteration Statement (Cont.)

- If you incorrectly specified the loop-continuation condition as `counter < 10` rather than `counter <= 10`, the loop would iterate only nine times.
- This is a common logic error called an off-by-one error.
- An incorrect final value in a loop-continuation condition also can cause an off-by-one error. In a loop that outputs 1 to 10, the loop-continuation condition should be `counter <= 10` rather than `counter < 10` (which causes an off-by-one error) or `counter < 11` (which is correct).
- Many programmers prefer zero-based counting in which, to count 10 times, `counter` would be initialized to zero, and the loop-continuation test would be `counter < 10`.

for Iteration Statement (Cont.)

- for Statement's Header

- The first line—including the keyword `for` and everything in the parentheses after `for`—is sometimes called the `for` statement header.
- The `for` header “does it all,” specifying each item needed for counter-controlled iteration.



for Iteration Statement (Cont.)

- General Format of a for Statement

- The general format of the for statement is

```
for (initialization; loopContinuationCondition; increment) {  
    statement(s)  
}
```

where

- ◆ initialization declares the loop's control variable and provides its initial value,
- ◆ loopContinuationCondition—between the two required semicolons—determines whether the loop should continue executing, and
- ◆ increment modifies the control variable's value so that the loop-continuation condition eventually becomes false.

for Iteration Statement (Cont.)

- General Format of a for Statement

- If the loopContinuationCondition is initially false, the program does not execute the for statement's body.
- The initialization and increment expressions can be **comma-separated lists** containing multiple initialization expressions or multiple increment expressions—the expressions in each evaluate left to right.
- Placing a semicolon immediately after the right parenthesis of a for header makes that for's body an empty statement. This usually is a logic error.
- **Infinite loops** occur when an iteration statement's loopContinuationCondition never becomes false.

for Iteration Statement (Cont.)

- Scope of a Statement's Control Variable
 - If the initialization expression declares the control variable, it can be used only in that for statement—not beyond it.
 - This restricted use is known as the **variable's scope**, which defines its lifetime and where it can be used in a program.
 - ◆ For example, a variable's scope is from its declaration point in a block to the right brace that closes the block. As you'll see in Chapter 5, it's good practice limit each variable's scope to only where it's needed.

for Iteration Statement (Cont.)

- Expressions in a for Statement's Header Are Optional
 - If you omit the loopContinuationCondition, the condition is always true, creating an **infinite loop**. Sometimes, this is desirable.
 - You might omit the initialization expression if the program initializes the control variable before the loop.
 - You might omit the increment expression if the program calculates the increment in the loop's body or if no increment is needed.

for Iteration Statement (Cont.)

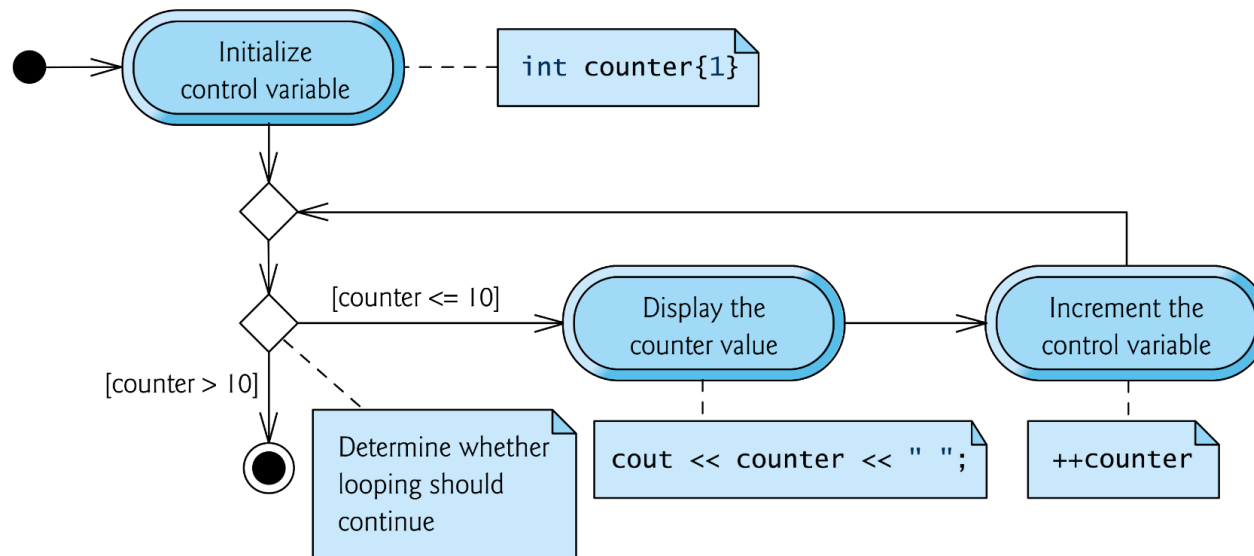
- Expressions in a for Statement's Header Are Optional
 - The increment expression in a for acts like a stand-alone statement at the end of the for's body.
 - Therefore, the following increment expressions are equivalent in a for:
counter = counter + 1
counter += 1
++counter
counter++
 - The increment expression does not appear in a larger expression, so preincrementing and postincrementing have the same effect.
 - ◆ We prefer preincrement. In Chapter 11's operator overloading discussion, you'll see that preincrement can have a performance advantage.

for Iteration Statement (Cont.)

- Using a for Statement's Control Variable in the Statement's Body
 - Programs frequently display the control variable's value or use it in calculations in the loop's body, but this is not required.
 - Also, the control variable's value can be changed in the loop's body, but doing so can lead to **subtle errors**.
 - If a program must modify the control variable's value in the loop's body, prefer **while** to for.

for Iteration Statement.

- UML Activity Diagram of the for Statement
 - It makes it clear that initialization occurs once—before the condition is tested the first time.
 - Incrementing occurs after the body statement executes.



Examples Using the for Statement

- The following examples show methods of varying the control variable in a for statement.

- Vary the control variable from 1 to 100 in increments of 1.

```
for (int i{1}; i <= 100; ++i)
```

- Vary the control variable from 100 to 1 in increments of -1

```
for (int i{100}; i >= 1; --i)
```

- Vary the control variable from 7 to 77 in steps of 7.

```
for (int i{7}; i <= 77; i += 7)
```

- Vary the control variable from 20 to 2 in steps of -2.

```
for (int i{10}; i >= 2; i -= 2)
```

- Vary the control variable over the following sequence of values: 2, 5, 8, 11, 14, 17.

```
for (int i{2}; j <= 17; j += 3)
```

- Vary the control variable over the following sequence of values: 44, 33, 22, 11, 0.

```
for (int i{44}; j >= 0; j -= 11)
```

Examples Using the for Statement.

- Do not use equality operators (`!=` or `==`) in a loop-continuation condition if the loop's control variable increments or decrements by more than 1.
- For example, in the following for statement header, `counter != 10` never becomes false, resulting in an infinite loop because `counter` increments by 2 after each iteration, producing only the odd values 3, 5, 7, 9, 11, ...:

```
for (int counter{1}; counter != 10; counter += 2)
```
- It's usually a logic error to use an incorrect relational operator in the loop-continuation condition of a loop that counts downward—for example, using `i <= 1` instead of `i >= 1` in a loop counting down to 1.

Application: Summing Even Integers; Introducing C++20 Text Formatting

- The next application uses a for statement to sum the even integers from 2 to 20 and store the result in int variable total.

```
1 // Summing integers with the for statement; introducing text formatting.
2 #include <format>
3 #include <iostream>
4 using namespace std;
5
6 int main() {
7     int total{0};
8
9     // total even integers from 2 through 20
10    for (int number{2}; number <= 20; number += 2) {
11        total += number;
12    }
13
14    cout << format("Sum is {}\n", total);
15 }
```

Application: Summing Even Integers; Introducing C++20 Text Formatting (Cont.)

- When lines 10–12 execute, each loop iteration adds control variable `number`'s value to `total`'s current value and stores the result in `total`.

```
1 // Summing integers with the for statement; introducing text formatting.
2 #include <format>
3 #include <iostream>
4 using namespace std;
5
6 int main() {
7     int total{0};
8
9     // total even integers from 2 through 20
10    for (int number{2}; number <= 20; number += 2) {
11        total += number;
12    }
13
14    cout << format("Sum is {}\n", total);
15 }
```

Application: Summing Even Integers; Introducing C++20 Text Formatting (Cont.)

- C++20: Text Formatting with Function format
 - C++20 introduced powerful new string-formatting capabilities via the format function (in header `<format>`).
 - These capabilities greatly simplify C++ formatting by using a syntax similar to formatting in Python, Microsoft's .NET languages (like C# and Visual Basic) and the newer language Rust.
 - We'll primarily use this new type of formatting going forward.
 - ◆ Chapter 19, Stream I/O & C++20 Text Formatting, presents old-style formatting details in case you work with legacy software that uses the old style. It also presents a deeper look at C++20 text-formatting features.

Application: Summing Even Integers; Introducing C++20 Text Formatting.

● Format String Placeholders

- Line 14 demonstrates the format function:

```
cout << format("Sum is {}\n", total);
```
- The first argument to format is a **format string** containing one or more **placeholders** delimited by curly braces ({ and }).
- The function replaces the placeholders with the values of the function's other arguments—in this case, `total`'s value.
 - ◆ Each placeholder may specify additional **formatting instructions**.
 - ◆ Placeholders are replaced left-to-right with the corresponding arguments that follow the format string.

Application: Compound-Interest Calculations; Introducing Format Specifiers

- Application: Compound-Interest Calculations

- Consider the following problem statement:

- ◆ A person invests \$1000.00 in a savings account yielding 5% interest. Assuming that all interest is left on deposit in the account, calculate and print the amount of money in the account at the end of each year for 10 years. Use the following formula for determining these amounts:

$$a = p(1 + r)^n$$

where

p is the original amount invested (i.e., the principal)

r is the annual interest rate

n is the number of years

a is the amount on deposit at the end of the nth year.

- This problem involves a loop that performs the indicated calculation for each of the 10 years the money remains on deposit.

Application: Compound-Interest Calculations; Introducing Format Specifiers (Cont.)

```
1 // Compound-interest calculations with for.
2 #include <format>
3 #include <iostream>
4 #include <cmath> // for pow function
5 using namespace std;
6
7 int main() {
8     double principal{1000.00}; // initial amount before interest
9     double rate{0.05}; // interest rate
10
11     cout << format("Initial principal: {:>7.2f}\n", principal)
12           << format("    Interest rate: {:>7.2f}\n", rate);
13
14     // display headers
15     cout << format("\n{ }{:>20}\n", "Year", "Amount on deposit");
16
17     // calculate amount on deposit for each of ten years
18     for (int year{1}; year <= 10; ++year) {
19         // calculate amount on deposit at the end of the specified
20         double amount{principal * pow(1.0 + rate, year)} ;
21
22         // display the year and the amount
23         cout << format("{:4d}{:20.2f}\n", year, amount);
24     }
25 }
```

Initial principal: 1000.00	
Interest rate: 0.05	
Year	Amount on deposit
1	1050.00
2	1102.50
3	1157.63
4	1215.51
5	1276.28
6	1340.10
7	1407.10
8	1477.46
9	1551.33
10	1628.89

Application: Compound-Interest Calculations; Introducing Format Specifiers (Cont.)

- Our solution uses a loop to perform the calculation for each of the 10 years the money remains on deposit.
- We use double values here for the monetary calculations, then discuss the problems with using floating-point types to represent monetary amounts.
 - Industrial-strength financial applications require precise, “to-the-penny” monetary calculations.
- Our Objects Natural Case Study will show how to reimplement this example using the Boost Multiprecision library to perform precise monetary calculations.

Application: Compound-Interest Calculations; Introducing Format Specifiers (Cont.)

- Lines 8–9 initialize double variable `principal` to 1000.00 and double variable `rate` to 0.05.
 - C++ treats floating-point literals like 1000.00 and 0.05 as type `double`.
 - Similarly, C++ treats whole numbers like 7 and -22 as type `int`.

```
8  double principal{1000.00}; // initial amount before interest
9  double rate{0.05}; // interest rate
```

Initial principal: 1000.00
Interest rate: 0.05

Application: Compound-Interest Calculations; Introducing Format Specifiers (Cont.)

● Formatting the Principal and Interest Rate

- In the example's format string, we used one empty placeholder ({}) to insert a variable's value at the placeholder's position in the string.
- Lines 11–12 format the principal and rate values using placeholders that concisely specify formatting instructions.
- The formatted results are shown in the first two lines of the output.

```
11 cout << format("Initial principal: {:>7.2f}\n", principal)  
12    << format("      Interest rate: {:>7.2f}\n", rate);
```

Application: Compound-Interest Calculations; Introducing Format Specifiers (Cont.)

- Formatting the Principal and Interest Rate

- Each format call in lines 11–12 uses the placeholder:

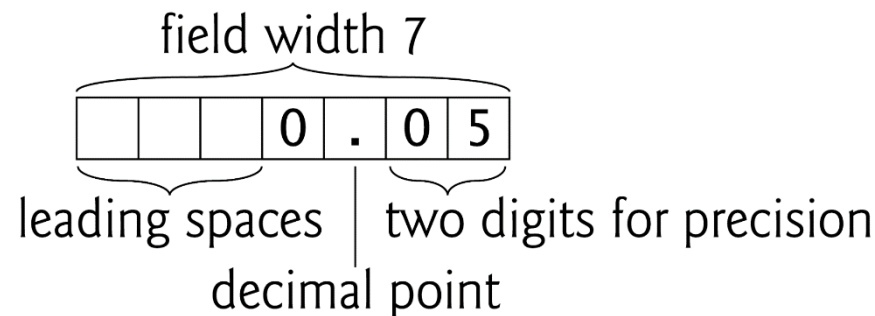
`{ :>7.2f }`

The colon (:) introduces a **format specifier** indicating how to format the corresponding value.

- In the format specifier `>7.2f` :
 - ◆ `>7` says the value should be right-aligned (`>`) in a field width of 7 character positions, and
 - ◆ `.2f` says we're formatting a floating-point number (`f`) with two digits of precision (`.2`) to the right of the decimal point.

Application: Compound-Interest Calculations; Introducing Format Specifiers (Cont.)

- Formatting the Principal and Interest Rate
 - The value of principal (1000.00) requires exactly seven characters to display, including the decimal point, so it fills the entire 7 character positions.
 - However, rate's value (0.05) requires only four total character positions, so it will be right-aligned in the 7 character positions and filled from the left with leading spaces, as in the following diagram:



Application: Compound-Interest Calculations; Introducing Format Specifiers (Cont.)

- Formatting the Principal and Interest Rate
 - Numeric values are right-aligned by default, so the `>` is not required here.
 - You also can left-align numeric values in a field width via `<`. If we were to do that here for 0.05, the value would be formatted with three trailing spaces.

Application: Compound-Interest Calculations; Introducing Format Specifiers (Cont.)

- Formatting the Year and Amount-on-Deposit Values in the for Loop

- Line 23's format string uses two placeholders to format the year and amount values: `"{:>4d}{:>20.2f}\n"`
 - ◆ The placeholder `{:>4d}` indicates that year's value should be formatted as an integer (d means decimal integer) right-aligned (>) in a **field of width 4**. This right-aligns all the year values under the four-character "Year" column.
 - ◆ The placeholder `{:>20.2f}` formats amount's value right-aligned (>) in a **field width of 20** as a floating-point number **with two digits to the right of the decimal point** (.2f). Formatting the amounts this way aligns their decimal points vertically, as is typical with monetary amounts. The field width of 20 right-aligns the amounts under the column heading "Amount on Deposit".

```
23 | cout << format("{:>4d}{:>20.2f}\n", year, amount);
```

Application: Compound-Interest Calculations; Introducing Format Specifiers (Cont.)

- Floating-Point Number Precision and Memory Requirements
 - A float represents a single-precision floating-point number.
 - ◆ Most of today's systems store these in four bytes of memory with approximately seven significant digits.
 - A double represents a double-precision floating-point number.
 - ◆ Most of today's systems store these in eight bytes of memory with approximately 15 significant digits—approximately double the precision of floats.
 - Most programmers use type double.
 - ◆ C++ treats floating-point numbers such as 3.14159 in a program's source code as **double** values **by default**. Such values in the source code are known as [floating-point literals](#).
 - ◆ The C++ standard requires only that type double provide at least as much precision as float. There is also type long double, which provides at least as much precision as double.

Application: Compound-Interest Calculations; Introducing Format Specifiers (Cont.)

- Floating-Point Numbers Are Approximations

- In conventional arithmetic, floating-point numbers often arise as a result of division. Dividing 10 by 3, the result is 3.3333333..., with the sequence of 3s repeating infinitely. The computer allocates a fixed amount of space to hold such a value, so the stored value can be only an approximation.
 - ◆ Floating-point types, like double, suffer from what is referred to as representational error. Assuming that floating-point numbers are represented exactly (e.g., using them in comparisons for equality) can lead to incorrect results.

Application: Compound-Interest Calculations; Introducing Format Specifiers (Cont.)

- Floating-Point Numbers Are Approximations

- Floating-point numbers have numerous applications, especially for measured values.
 - ◆ For example, when we speak of a “normal” body temperature of 98.6 degrees Fahrenheit, we do not need to be precise to a large number of digits.
 - ◆ When we read the temperature on a thermometer as 98.6, it actually might be 98.594732103. Calling this number 98.6 is fine for most body temperature calculations.
 - ◆ Generally, double is preferred over float because doubles represent floating-point numbers more precisely.²⁶²

Application: Compound-Interest Calculations; Introducing Format Specifiers (Cont.)

- A Warning about Displaying Rounded Values

- Here is a simple explanation of what can go wrong when using float or double to represent dollar amounts.

- ◆ Two float dollar amounts stored in the machine could be 14.234 (which prints as 14.23) and 18.673 (which prints as 18.67).

- ◆ When these amounts are added, they produce the sum 32.907, which prints as 32.91.

- ◆ Thus your printout could appear as

$$\begin{array}{r} 14.23 \\ + 18.67 \\ \hline 32.91 \end{array}$$

- ◆ Clearly the sum of the individual numbers as printed should be 32.90! You've been warned!

Application: Compound-Interest Calculations; Introducing Format Specifiers.

- Even Common Monetary Amounts Can Have Floating-Point Representational Errors
 - Even simple monetary amounts stored as doubles can have representational errors.
 - To see this, we created a simple program that defined the variable `d` as follows:

```
double d{123.02};
```
 - We displayed `d`'s value with 20 digits of precision to the right of the decimal point. The resulting output showed 123.02 as 123.0199999..., which is another example of a representational error.
 - Though some monetary amounts can be represented precisely as doubles, many cannot. In Objects Natural Case Study, we represent monetary amounts precisely with the Boost Multiprecision library.

do...while Iteration Statement

- In a while statement, the program tests the loop-continuation condition before executing the loop's body.
 - If it's false, the body never executes.
- The do...while iteration statement tests the loop-continuation condition after executing the loop's body,
 - So the body always executes at least once.

do...while Iteration Statement (Cont.)

- Next program uses a do...while to output the numbers 1–10.
 - Line 6 declares and initializes control variable counter.
 - Upon entering the do...while statement, line 9 outputs counter's value, and line 10 increments counter.

```
1 // do...while iteration statement.
2 #include <iostream>
3 using namespace std;
4
5 int main() {
6     int counter{1};
7
8     do {
9         cout << counter << " ";
10        ++counter;
11    } while (counter <= 10); // end do...while
12
13    cout << "\n";
14 }
```


do...while Iteration Statement (Cont.)

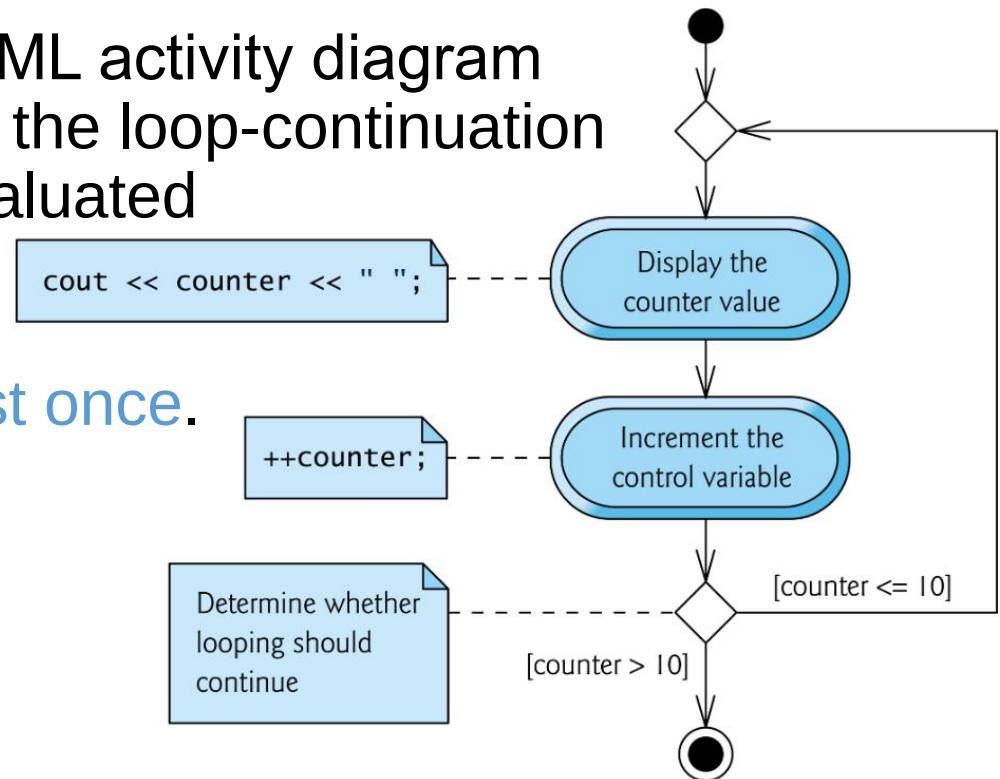
- Then the program evaluates the loop-continuation test at the bottom of the loop (line 11).
 - ◆ If the condition is true, the loop continues at the first body statement (line 9).
 - ◆ If the condition is false, the loop terminates, and the program continues at the next statement after the loop.

```
1 // do...while iteration statement.
2 #include <iostream>
3 using namespace std;
4
5 int main() {
6     int counter{1};
7
8     do {
9         cout << counter << " ";
10        ++counter;
11    } while (counter <= 10); // end do...while
12
13    cout << "\n";
14 }
```

do...while Iteration Statement.

- UML Activity Diagram for the do...while Iteration Statement

- The do...while's UML activity diagram makes it clear that the loop-continuation condition is not evaluated until after the loop performs the action state **at least once**.



`switch` Multiple-Selection Statement

- C++ provides the `switch` **multiple-selection statement** to choose among many different actions based on the possible values of a variable or expression.
- Each action is associated with the value of an integral constant expression—that is, any combination of **character** and **integer constants** that evaluates to a **constant integer value**.

`switch` Multiple-Selection Statement (Cont.)

- Using a `switch` Statement to Count A, B, C, D and F Grades
 - The next program calculates the class average of a set of numeric grades entered by the user.
 - The switch statement determines each grade's letter equivalent (A, B, C, D or F) and increments the appropriate grade counter.
 - The program also summarizes the number of students who received each grade.

switch Multiple-Selection Statement (Cont.)

- The program declares local variables `total` (line 7) and `gradeCounter` (line 8) to keep track of the sum of the grades entered by the user and the number of grades entered.
- Lines 9–13 declare counter variables for each grade category, initializing each to 0.

```
1 // Using a switch statement to count letter grades.
2 #include <format>
3 #include <iostream>
4 using namespace std;
5
6 int main() {
7     double total{0.0}; // sum of grades
8     int gradeCounter{0}; // number of grades entered
9     int aCount{0}; // count of A grades
10    int bCount{0}; // count of B grades
11    int cCount{0}; // count of C grades
12    int dCount{0}; // count of D grades
13    int fCount{0}; // count of F grades
14
15    cout << "Enter the integer grades in the range 0-100.\n"
16         << "Type the end-of-file indicator to terminate input:\n"
17         << "    On UNIX/Linux/macOS type <Ctrl> d then press Enter\n"
18         << "    On Windows type <Ctrl> z then press Enter\n";
```

switch Multiple-Selection Statement (Cont.)

- Lines 23–50 input an arbitrary number of integer grades using sentinel-controlled iteration, update variables `total` and `gradeCounter`, and increment an appropriate letter-grade counter for each grade entered.

```
20
21
22
23 while (cin >> grade) {
24     total += grade; // add grade to total
25     ++gradeCounter; // increment number of grades
26
27     // increment appropriate letter-grade counter
28     switch (grade / 10) {
29         case 9: // grade was between 90
30             case 10: // and 100, inclusive
31                 ++aCount;
32                 break; // exits switch
33
34         case 8: // grade was between 80 and 89
35             ++bCount;
36             break; // exits switch
37
38         case 7: // grade was between 70 and 79
39             ++cCount;
40             break; // exits switch
41
42         case 6: // grade was between 60 and 69
43             ++dCount;
44             break; // exits switch
45
46         default: // grade was less than 60
47             ++fCount;
48             break; // optional; exits switch anyway
49     } // end switch
50 } // end while
```

switch Multiple-Selection Statement (Cont.)

- Lines 53–70 output a report containing the total of all grades entered, the average grade and the number of students who received each letter grade.

```
52 // display grade report
53 cout << "\nGrade Report:\n";
54
55 // if user entered at least one grade...
56 if (gradeCounter != 0) {
57     // calculate average of all grades entered
58     double average{total / gradeCounter};
59
60     // output summary of results
61     cout << format("Total of the {} grades entered is {}\n",
62                   gradeCounter, total)
63           << format("Class average is {:.2f}\n\n", average)
64           << "Summary of student's grades:\n"
65           << format("A: {}\nB: {}\nC: {}\nD: {}\nF: {}\n",
66                   aCount, bCount, cCount, dCount, fCount);
67 }
68 else { // no grades were entered, so output appropriate message
69     cout << "No grades were entered\n";
70 }
71 }
```

switch Multiple-Selection Statement (Cont.)

Enter the integer grades in the range 0-100.
Type the end-of-file indicator to terminate input:
On UNIX/Linux/macOS type <Ctrl> d then press Enter
On Windows type <Ctrl> z then press Enter

99
92
45
57
63
71
76
85
90
100
^Z

Grade Report:
Total of the 10 grades entered is 778
Class average is 77.80

Summary of student
A: 4
B: 1
C: 2
D: 1
F: 2

Possible results

switch Multiple-Selection Statement (Cont.)

- Reading Grades from the User

- Lines 15–18 prompt the user to enter integer grades or type the **end-of-file indicator** to terminate the input.
 - ◆ The end-of-file indicator is a system-dependent keystroke combination that indicates there's no more data to input.
 - ◆ In Chapter 8, you'll see how the end-of-file indicator is used when a program reads its input from a file.

```
15      cout << "Enter the integer grades in the range 0-100.\n"
16      << "Type the end-of-file indicator to terminate input:\n"
17      << "    On UNIX/Linux/macOS type <Ctrl> d then press Enter\n"
18      << "    On Windows type <Ctrl> z then press Enter\n";
```

switch Multiple-Selection Statement (Cont.)

- Reading Grades from the User

- The keystroke combinations for entering end-of-file are system dependent.

- ◆ On UNIX/Linux/macOS systems, type the following key sequence on a line by itself:

<Ctrl> d

This notation means to press both the Ctrl key and the d key simultaneously—on some keyboards Ctrl is labeled as control.

- ◆ On Windows systems, type

<Ctrl> z

- ◆ On some systems, you must also press Enter.
- ◆ Also, Windows typically displays ^Z on the screen when you type the end-of-file indicator.

switch Multiple-Selection Statement (Cont.)

- Reading Grades from the User

- The `while` statement (lines 23–50) obtains the user input.
- Line 23 performs the input in the `while` statement's condition.
 - ◆ The loop-continuation condition evaluates to true if `cin` successfully reads an `int` value.
 - ◆ If the user enters the end-of-file indicator, the condition evaluates to false.
- If the condition is true, line 24 adds `grade` to `total`, and line 25 increments `grade-Counter`. These are used to compute the average.

```
23 while (cin >> grade) {  
24     total += grade; // add grade to total  
25     ++gradeCounter; // increment number of grades
```

switch Multiple-Selection Statement (Cont.)

- Reading Grades from the User

- Next, lines 28–49 use a switch statement to increment the appropriate letter-grade counter based on the numeric grade entered.

```
28 switch (grade / 10) {
29     case 9: // grade was between 90
30     case 10: // and 100, inclusive
31         ++aCount;
32         break; // exits switch
33
34     case 8: // grade was between 80 and 89
35         ++bCount;
36         break; // exits switch
37
38     case 7: // grade was between 70 and 79
39         ++cCount;
40         break; // exits switch
41
42     case 6: // grade was between 60 and 69
43         ++dCount;
44         break; // exits switch
45
46     default: // grade was less than 60
47         ++fCount;
48         break; // optional; exits switch anyway
49 } // end switch
```

switch Multiple-Selection Statement (Cont.)

- Processing the Grades

- The `switch` statement (lines 28–49) determines which counter to increment. We assume that the user enters a valid grade in the range 0–100 where:

- ◆ 90–100 represents A,
- ◆ 80–89 represents B,
- ◆ 70–79 represents C,
- ◆ 60–69 represents D, and
- ◆ 0–59 represents F.

```
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49

switch (grade / 10) {
    case 9: // grade was between 90
            ++aCount;
            break; // exits switch

    case 8: // grade was between 80 and 89
            ++bCount;
            break; // exits switch

    case 7: // grade was between 70 and 79
            ++cCount;
            break; // exits switch

    case 6: // grade was between 60 and 69
            ++dCount;
            break; // exits switch

    default: // grade was less than 60
            ++fCount;
            break; // optional; exits switch anyway
} // end switch
```

`switch` Multiple-Selection Statement (Cont.)

- Processing the Grades.

- The switch statement's block contains a sequence of `case labels` and an `optional default case`, which can appear anywhere in the switch, but typically appears last.
- When the flow of control reaches the `switch`, the program evaluates the controlling expression `grade / 10` in the parentheses following keyword `switch`, then compares this value with each case label.
- The expression must have an integral type—`bool`, `char`, `char8_t`, `char16_t`, `char32_t`, `wchar_t`, `int`, `long` **or** `long long`.

switch Multiple-Selection Statement (Cont.)

- Controlling Expression Produces Integer Values
 - Line 28's controlling expression performs integer division, which truncates the result's fractional part.

```
28  switch (grade / 10) {
```

- ◆ When we divide an int value from 0 to 100 by 10, the result is always an int value from 0 to 10.
- ◆ We use several of these values in our [case labels](#).
- ◆ If the user enters the integer 85, the controlling expression evaluates to 8.

switch Multiple-Selection Statement (Cont.)

- Controlling Expression Produces Integer Values
 - The switch compares 8 with each case label.
 - ◆ If a match occurs (case 8: at line 34), that case's statements execute. For 8, line 36 increments `bCount`, because a grade in the 80s is a B.
 - ◆ The `break` statement (line 36) exits the switch.

```
34      case 8: // grade was between 80 and 89
35          ++bCount;
36          break; // exits switch
```


`switch` Multiple-Selection Statement (Cont.)

- Performing the Same Statements for Multiple cases
 - Our `switch` statement's cases explicitly test for the values 10, 9, 8, 7 and 6.
 - ◆ Note the cases in lines 30–31 that test for the values 9 and 10 (both of which represent the grade A).
 - ◆ Listing cases consecutively with no statements between them enables the cases to perform the same set of statements.
 - So, when the controlling expression evaluates to 9 or 10, the statements in lines 31–32 execute.
 - ◆ The `switch` statement does not provide a mechanism for testing ranges of values, so every value you need to test must be listed in a separate case label.
 - ◆ Each case can have **multiple** statements.
 - The `switch` statement differs from other control statements in that it does **not require braces** around multiple statements in a case unless you need to declare a variable in a case.

`switch` Multiple-Selection Statement (Cont.)

- case without a `break` Statement—
[[fallthrough]] Attribute
 - Generally, every case ends with a `break` statement.
 - Without `break` statements, each time a match occurs in the `switch`, the statements for that case and subsequent cases execute until a `break` statement or the end of the `switch` is reached.
 - This is referred to as “falling through” to the statements in subsequent cases.

switch Multiple-Selection Statement (Cont.)

- case without a `break` Statement—`[[fallthrough]]` Attribute
 - Forgetting a `break` statement when one is needed is a logic error.
 - ◆ To call your attention to this possible problem, many compilers issue a warning when a case label is followed by one or more statements and does not contain a `break` statement.
 - For such instances in which “falling through” is the desired behavior, you can use the attribute `[[fallthrough]]` to tell the compiler that “falling through” to the next case is the correct behavior by placing the statement `[[fallthrough]];` where the `break` statement would normally appear.

switch Multiple-Selection Statement (Cont.)

- The default Case

- If the controlling expression's value does not match any of the case labels, the default case (lines 46–48) executes.
- We use the default case in this example to process all controlling-expression values that are less than 6—that is, all failing grades.
- If no match occurs and the switch does not contain a default case, program control simply continues with the first statement after the switch.
- In a switch, it's good practice to test all possible values of the controlling expression.
- Also, providing a default case focuses you on the need to process **exceptional conditions**.

```
46         default: // grade was less than 60
47             ++fCount;
48             break; // optional; exits switch anyway
```

switch Multiple-Selection Statement (Cont.)

- Displaying the Grade Report

- Lines 53–70 output a report based on the grades entered.
- Line 56 determines whether the user entered at least one grade.
 - ◆ This helps us **avoid dividing by zero**, which for integer division causes the program to fail and for floating-point division produces the value **nan** (“**not a number**”).
- If no grades were entered, line 69 outputs an appropriate message.

```
56  if (gradeCounter != 0) {  
68  else { // no grades were entered, so output appropriate message  
69      cout << "No grades were entered\n";  
70  }  
71 }
```

switch Multiple-Selection Statement (Cont.)

- Displaying the Grade Report

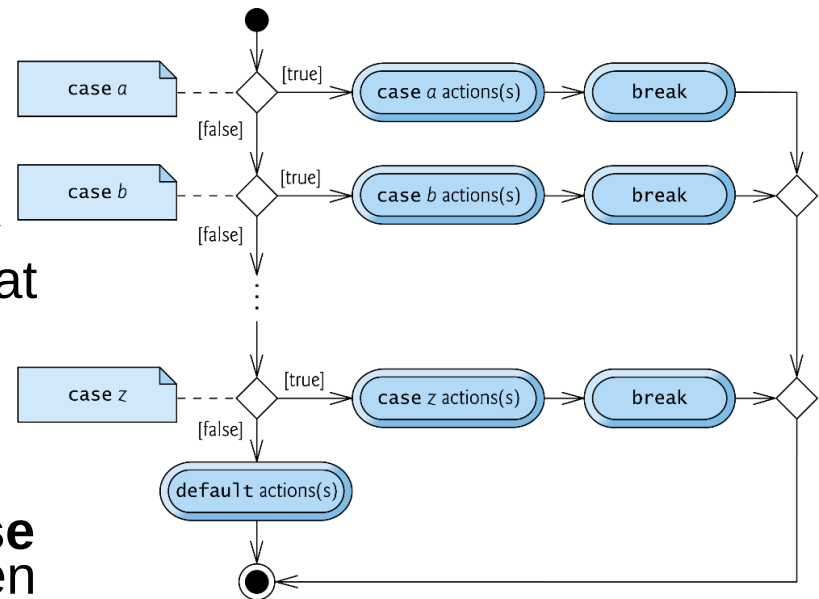
- If any grade is entered, line 58 calculates the average of the grades—recall that a double divided by an int produces a double result.
- Lines 61–66 then output the total of all the grades, the class average and the number of students who received each letter grade.

```
58         double average{total / gradeCounter};
59
60         // output summary of results
61         cout << format("Total of the {} grades entered is {}\n",
62                        gradeCounter, total)
63                << format("Class average is {:.2f}\n\n", average)
64                << "Summary of student's grades:\n"
65                << format("A: {}\nB: {}\nC: {}\nD: {}\nF: {}\n",
66                        aCount, bCount, cCount, dCount, fCount);
```

switch Multiple-Selection Statement (Cont.)

switch Statement UML Activity Diagram

- Most switch statements use a **break** in each case to terminate the switch after the case is processed.
- The diagram emphasizes this by including **breaks** and showing that the break at the end of a case causes control to exit the switch immediately.
- The break statement is not required for the switch's **last case** or the optional **default case** when it appears last because execution continues with the next statement after the switch.



`switch` Multiple-Selection Statement.

- Notes on cases

- Each case in a `switch` statement must contain a **constant integral expression**—that is, any expression that evaluates to a constant integer value.
- You can also use `enum` constants (introduced in Section 5.9) and character literals—specific characters in single quotes, such as `'a'`, `'A'` or `'$'`, which represent the integer values of characters.
- In Chapter 10, we present a more elegant way to implement `switch` logic. We use a technique called **polymorphism** to create programs that are often clearer, easier to maintain and extend than programs using `switch` logic.

Selection Statements with Initializers

- Earlier, we introduced the for iteration statement. In the for header's initialization section, we declared and initialized a control variable, which limited that variable's scope to the for statement.
- Selection statements with initializers enable you to include variable initializers before the condition in an if or if...else statement and before the controlling expression of a switch statement.
- As with the for statement, these variables are known only in the statements where they're declared.

Selection Statements with Initializers (Cont.)

- The next program shows if...else statements with initializers (lines 7–12 and 14–19).

```
1 // if statements with initializers.
2 #include <format>
3 #include <iostream>
4 using namespace std;
5
6 int main() {
7     if (int value{7}; value == 7) {
8         cout << format("value is {}\n", value);
9     }
10    else {
11        cout << format("value is not 7; it is {}\n", value);
12    }
13
14    if (int value{13}; value == 9) {
15        cout << format("value is {}\n", value);
16    }
17    else {
18        cout << format("value is not 9; it is {}\n", value);
19    }
20 }
```

Selection Statements with Initializers (Cont.)

- Syntax of Selection Statements with Initializers
 - For an if or if...else statement, you place the initializer first in the condition's parentheses.
 - For a switch statement, you place the initializer first in the controlling expression's parentheses.
 - The initializer must end with a semicolon (;), as in lines 7 and 14.
 - The initializer can declare multiple variables of the same type in a comma-separated list.

Selection Statements with Initializers.

- Scope of Variables Declared in the Initializer
 - Any variable declared in the initializer of an if, if...else or switch statement may be used throughout the remainder of the statement.
 - In lines 7–12, we use the variable value to determine which branch of the if...else statement to execute, then use value in the output statements of both branches.
 - When the if...else statement terminates, value no longer exists, so we can use that identifier again in the second if...else statement to declare a new variable known only in that statement.

break and continue Statements

- In addition to selection and iteration statements, C++ provides break and continue statements to alter the flow of control.
- The preceding section showed how `break` could be used to terminate a switch statement's execution.
- This section discusses how to use break in **iteration** statements.

break and continue Statements (Cont.)

- break Statement

- Executing a break statement in a while, for, do...while or switch causes immediate exit from that statement—execution continues with the first statement after the control statement.
- Common uses of break include escaping early from a loop or exiting a switch.

break and continue Statements (Cont.)

- break Statement

- When the if statement nested at lines 9–11 in the for statement (lines 8–14) detects that count is 5, the break statement at line 10 executes.

```
1 // break statement exiting a for statement.
2 #include <iostream>
3 using namespace std;
4
5 int main() {
6     int count; // control variable also used after loop
7
8     for (count = 1; count <= 10; ++count) { // loop 10 times
9         if (count == 5) {
10             break; // terminates for loop if count is 5
11         }
12
13         cout << count << " ";
14     }
15
16     cout << "\nBroke out of loop at count = " << count << "\n";
17 }
```

1 2 3 4
Broke out of loop at count = 5

break and continue Statements (Cont.)

- break Statement.

- This terminates the for statement, and the program proceeds to line 16 (immediately after the for statement), which displays a message indicating the control variable's value when the loop terminated.

```
1 // break statement exiting a for statement.
2 #include <iostream>
3 using namespace std;
4
5 int main() {
6     int count; // control variable also used after loop
7
8     for (count = 1; count <= 10; ++count) { // loop 10 times
9         if (count == 5) {
10             break; // terminates for loop if count is 5
11         }
12
13         cout << count << " ";
14     }
15
16     cout << "\nBroke out of loop at count = " << count << "\n";
17 }
```


break and continue Statements (Cont.)

- `continue` Statement

- Executing the `continue` statement in a `while`, `for` or `do...while` **skips the remaining statements** in the loop body and proceeds with the next iteration of the loop.
- In `while` and `do...while` statements, the program evaluates the loop-continuation test immediately after the `continue` statement executes.
- In a `for` statement, the increment expression executes, then the program evaluates the loop-continuation test.

break and continue Statements (Cont.)

- `continue` Statement.

- The next program uses `continue` (line 8) to skip the statement at line 11 when the nested `if` determines that `count`'s value is 5.
 - ◆ When the `continue` statement executes, program control continues with the increment of the control variable in the `for` statement (line 6).

```
1 // continue statement terminating an iteration of a for statement.
2 #include <iostream>
3 using namespace std;
4
5 int main() {
6     for (int count{1}; count <= 10; ++count) { // loop 10 times
7         if (count == 5) {
8             continue; // skip remaining code in loop body if count is 5
9         }
10
11         cout << count << " ";
12     }
13
14     cout << "\nUsed continue to skip printing 5\n";
15 }
```

1 2 3 4 6 7 8 9 10 Used continue to skip printing 5
--

break and continue Statements.

- Software Engineering vs. Performance Trade-Off
 - Some programmers feel that `break` and `continue` **violate** structured programming.
 - ◆ Since the same effects are achievable with structured-programming techniques, these programmers prefer to avoid `break` and `continue` (but at a slight decrease in performance).
 - There's a tension between achieving quality software engineering and achieving the best-performing software.
 - Sometimes one of these goals is achieved at the expense of the other.
 - For all but the most performance-intensive situations, you should first make your code simple and correct, then make it fast and small—but only if necessary.

Logical Operators

- The conditions in `if`, `if...else`, `while`, `do...while` and `for` statements determine how to continue a program's flow of control.
- So far, we've presented simple conditions, such as `count <= 10`, `number != sentinelValue` and `total > 1000`.
- You express simple conditions with relational (`>`, `<`, `>=` and `<=`) and equality (`==` and `!=`) operators.
- Sometimes control statements require more complex conditions to determine a program's flow of control.
- C++'s logical operators enable you to combine simple conditions. The logical operators are `&&` (logical AND), `||` (logical OR) and `!` (logical negation).

Logical Operators (Cont.)

- Logical AND (& &) Operator

- In 2023, after three years of the Covid pandemic, the United States experienced a pilot shortage that snarled air traffic. To alleviate the pilot shortage, the government considered raising the mandatory pilot retirement age from 65 to 67, so senior pilots in good health could continue flying for two more years.
- Let's assume that one airline wanted to ask each senior pilot aged 63 or older whether they'd like to apply to keep flying until age 67 rather than retiring at age 65. The airline could write a program to search their computerized employee records and determine the number of 63-years-of-age-or-older senior pilots.

Logical Operators (Cont.)

- Logical AND (&&) Operator

- The program could contain the statement:

```
if (employeeType == PILOT && age >= 63) {  
    ++seniorPilots;  
}
```

- Assume PILOT is a constant. The if statement contains two simple conditions:

- ◆ `employeeType == PILOT` determines whether an employee is a pilot.
- ◆ `age >= 63` determines whether an employee is nearing retirement age.

Logical Operators (Cont.)

- Logical AND (&&) Operator

- The if statement considers the combined condition, **which is true only if both simple conditions are true.** If so, the if statement's body increments `seniorPilots` by 1; otherwise, the program skips the increment. Some programmers find that the preceding combined condition is more readable with redundant parentheses, as in

```
if ((employeeType == PILOT) && (age >= 63)) {  
    ++seniorPilots;  
}
```

Logical Operators (Cont.)

- Logical AND (& &) Operator.

- The following truth table summarizes the & & operator, showing the bool values false and true combinations for expression1 and expression2.
- C++ evaluates to false or true all expressions that include relational operators, equality operators or logical operators:

Logical AND (&&) Operator Truth Table		
expression1	expression2	expression1 && expression2
false	false	false
false	true	false
true	false	false
true	true	true

Logical Operators (Cont.)

- Logical OR (||) Operator

- Now suppose we wish to ensure that **either or both of two conditions are true** before we choose a certain path of execution. In this case, we use the || (**logical OR**) operator, as in:

```
if ((semesterAverage >= 90) || (finalExam >= 90)) {  
    cout << "Student grade is A\n";  
}
```

- This statement also contains two simple conditions:

- ◆ `semesterAverage >= 90` determines whether the student deserves an A in the course for a solid performance throughout the semester.
- ◆ `finalExam >= 90` determines whether the student deserves an A in the course for outstanding performance on the final exam.

Logical Operators (Cont.)

- Logical OR (||) Operator.

- The if statement then considers the combined condition

```
(semesterAverage >= 90) || (finalExam >= 90)
```

and awards the student an A if either or both of the simple conditions are true.

- The only time this would not print "Student grade is A" is if both simple conditions are false.

Logical Operators (Cont.)

- Logical OR (| |) Operator.

- The following is the truth table for the operator logical OR (| |):

Logical OR () Operator Truth Table		
expression1	expression2	expression1 expression2
false	false	false
false	true	true
true	false	true
true	true	true

- Operator & & has higher precedence than operator | |. Both operators group left to right. In general, use **parentheses** if there is ambiguity about evaluation order.

Logical Operators (Cont.)

- Short-Circuit Evaluation

- The parts of an expression containing && or || operators are evaluated only **until it's known whether the condition is true or false.**
- Thus, the evaluation of the condition
`(employeeType == PILOT) && (age >= 63)`
stops immediately if employeeType is not equal to PILOT (i.e., the entire expression is false) and continues if employeeType is equal to PILOT.
- This short-circuit evaluation is a performance feature of logical AND and logical OR expressions.

Logical Operators (Cont.)

- Short-Circuit Evaluation.

- In expressions using operator `&&`, a condition—we'll call this the dependent condition—may require another condition to be true for the evaluation of the dependent condition to be meaningful. In this case, the dependent condition should be placed after the `&&` operator to prevent errors.
- Consider the following condition:

```
(i != 0) && (10 / i == 2)
```

The dependent condition `(10 / i == 2)` must appear after the `&&` operator to prevent the possibility of division by zero.

Logical Operators (Cont.)

- Logical Negation (!) Operator

- The ! **logical negation operator** (also called the logical NOT or logical complement operator) “reverses” a condition’s meaning.
- The logical negation operator is a unary operator that has only one condition as an operand. To execute code only when a condition is false, place ! before the original condition, as in:

```
if (!(grade == sentinelValue)) {  
    cout << "The next grade is " << grade << "\n";  
}
```

- This executes the body statement only if grade is not equal to `sentinelValue`. The parentheses around the condition `grade == sentinelValue` are needed because the logical negation operator has higher precedence than the equality operator.

Logical Operators (Cont.)

- Logical Negation (!) Operator

- In most cases, you can avoid using logical negation by expressing the condition with an appropriate relational or equality operator.
- For example, the previous statement is more readable with the `!=` operator:

```
if (grade != sentinelValue) {  
    cout << "The next grade is " << grade << "\n";  
}
```

Logical Operators (Cont.)

- Logical Negation (!) Operator.
 - The following is the truth table for the ! logical negation operator:

Logical Negation (!) Operator Truth Table	
expression	!expression
false	true
true	false

Logical Operators (Cont.)

- Example: Producing Logical-Operator Truth Tables
 - The next program produces the logical operators' truth tables for `&&`, `||` and `!` (lines 8–12, 15–19 and 22–24).
 - The output shows each expression that's evaluated and its bool result.
 - ◆ By default, bool values true and false are displayed by `cout` and the stream-insertion operator as 1 and 0 but the `format` function displays the word true or the word false.

Logical Operators (Cont.)

```
1 // Logical operators.
2 #include <format>
3 #include <iostream>
4 using namespace std;
5
6 int main() {
7     // create truth table for && (logical AND) operator
8     cout << "Logical AND (&&)\n"
9         << format("false && false: {}\n", false && false)
10        << format("false && true: {}\n", false && true)
11        << format("true && false: {}\n", true && false)
12        << format("true && true: {}\n\n", true && true);
13
14     // create truth table for || (logical OR) operator
15     cout << "Logical OR (||)\n"
16         << format("false || false: {}\n", false || false)
17         << format("false || true: {}\n", false || true)
18         << format("true || false: {}\n", true || false)
19         << format("true || true: {}\n\n", true || true);
20
21     // create truth table for ! (logical negation) operator
22     cout << "Logical negation (!)\n"
23         << format("!false: {}\n", !false)
24         << format("!true: {}\n", !true);
25 }
```

Logical AND (&&)
false && false: false
false && true: false
true && false: false
true && true: true

Logical OR (||)
false || false: false
false || true: true
true || false: true
true || true: true

Logical negation (!)
!false: true
!true: false

Logical Operators.

- Precedence and Grouping of the Operators Presented So Far

Operators	Grouping
++ (<i>postfix</i>) -- (<i>postfix</i>)	left to right
+ - ! ++ (<i>prefix</i>) -- (<i>prefix</i>)	right to left
* / %	left to right
+ -	left to right
<< >>	left to right
< <= > >=	left to right
== !=	left to right
&&	left to right
	left to right
?:	right to left
= += -= *= /= %=	right to left

Confusing the Equality (==) and Assignment (=) Operators

- There's one logic error that C++ programmers, no matter how experienced, tend to make so frequently that we feel it requires a separate section.
- That error is accidentally swapping the operators == (equality) and = (assignment).
- What makes this so damaging is that it ordinarily does not cause compilation errors.
 - Statements with these errors tend to compile correctly and run to completion, often generating incorrect results through runtime logic errors.
 - Today's compilers generally issue warnings when = is used in contexts where == is expected.

Confusing the Equality (==) and Assignment (=) Operators (Cont.)

- Two aspects of C++ contribute to these problems.
 - One is that any expression that produces a value can be used in the decision portion of any control statement. If the expression's value is zero, it's treated as false. If the value is nonzero, it's treated as true.
 - The second is that assignments produce a value—namely, the value of the variable on the assignment operator's left side.

Confusing the Equality (==) and Assignment (=) Operators (Cont.)

- For example, suppose we intend to write

```
if (payCode == 4) { // good
    cout << "You get a bonus!\n";
}
```

but we accidentally write

```
if (payCode = 4) { // bad
    cout << "You get a bonus!\n";
}
```

This if statement contains a logic error that evaluates the assignment in the if condition to the value 4.

- Any nonzero value is true, so this condition always evaluates as true, and the person always receives a bonus regardless of the pay code!
- Even worse, the pay code has been modified when it was only supposed to be examined!

Confusing the Equality (==) and Assignment (=) Operators (Cont.)

- lvalues and rvalues

- You can prevent this problem with a simple trick.
- First, knowing what's allowed to the left of an assignment operator is helpful.
- Variable names are said to be lvalues (for “left values”) because they can be used on an assignment operator’s left side.
- Literals are said to be rvalues (for “right values”)—they can be used on only an assignment operator’s right side.
- You can use lvalues as rvalues on an assignment’s right side, but not vice versa.

Confusing the Equality (==) and Assignment (=) Operators (Cont.)

- lvalues and rvalues.

- Programmers usually write conditions like `x == 7` with the variable name (an lvalue) on the left and the literal (an rvalue) on the right.
- Placing the literal on the left, as in `7 == x` (which is syntactically correct and is sometimes called a “Yoda condition”), enables the compiler to issue an error if you accidentally replace the `==` operator with `=`.
 - ◆ The compiler treats this as a compilation error because you can’t change a literal’s value.

Confusing the Equality (==) and Assignment (=) Operators (Cont.)

- Using == in Place of =

- There's another equally unpleasant situation. Suppose you want to assign a value to a variable with a simple statement like

`x = 1;`

but instead write

`x == 1;`

- Here, too, this is not a syntax error. Rather, the compiler simply evaluates the expression
 - ◆ If x is equal to 1, the condition is true, and the expression evaluates to true.
 - ◆ If x is not equal to 1, the condition is false, and the expression evaluates to false.
 - ◆ Regardless of the expression's value, there's no assignment operator, so the value is lost. The value of x remains unaltered, probably causing an execution-time logic error.
- Using operator == for assignment and using operator = for equality are logic errors.

Confusing the Equality (==) and Assignment (=) Operators.

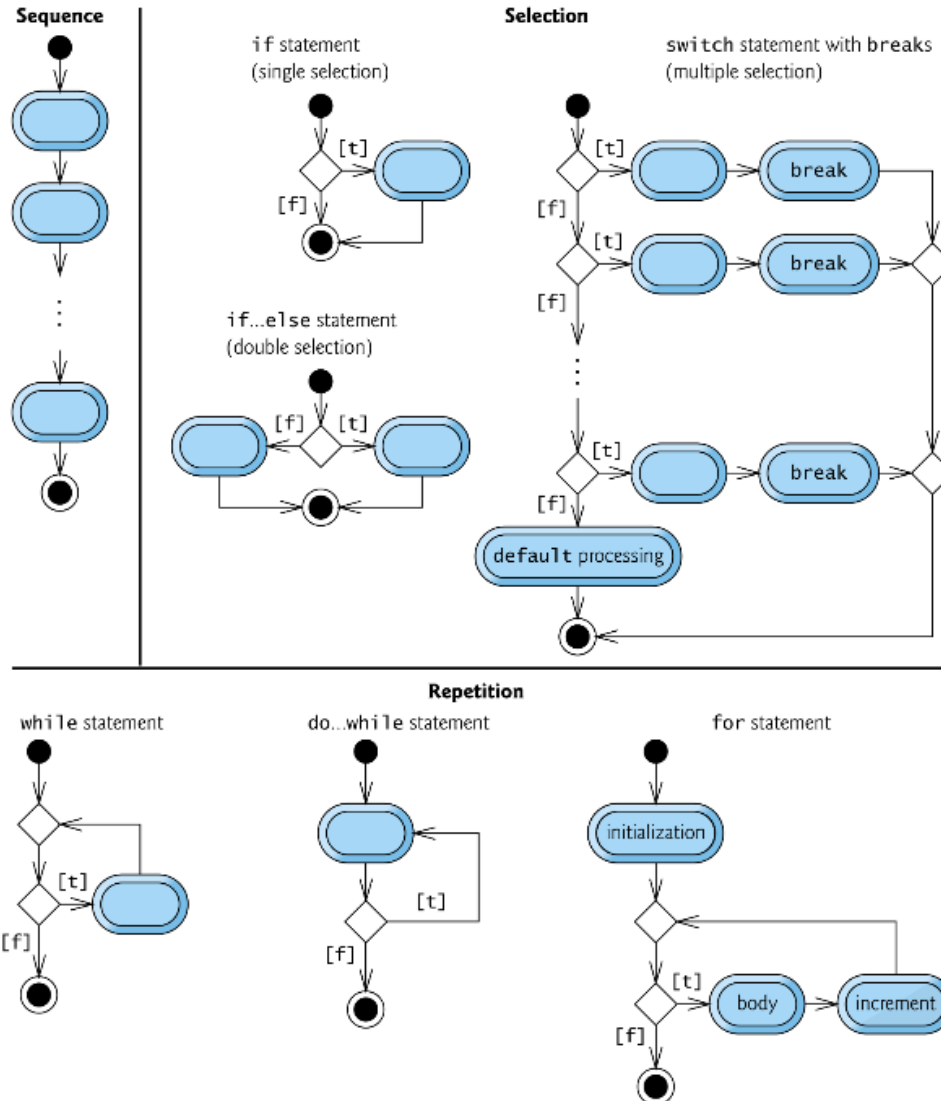
● Enabling Warnings

- Some compilers require you to enable warnings before they'll issue warning messages.
 - ◆ For g++ and clang++, add the -Wall (enable all warnings) flag to your compilation command.
 - ◆ For Visual C++:
 - In your solution, right-click the project's name and select Properties.
 - Expand Code Analysis and select General.
 - For Enable Code Analysis on Build, select Yes, then click OK.
- This might cause many warnings to be displayed for certain libraries.

Structured-Programming Summary

- Just as architects design buildings by employing the collective wisdom of their profession, so should programmers design programs.
- Our field is much younger than architecture, and our collective wisdom is considerably sparser.
- We've learned that structured programming produces programs that are easier than unstructured programs to understand, test, debug, modify and even prove correct in a mathematical sense.

Structured-Programming Summary (Cont.)



Structured-Programming Summary (Cont.)

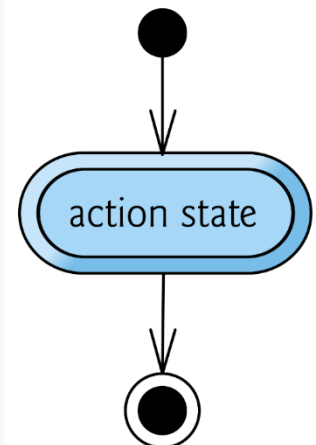
- C++ Control Statements Are Single-Entry/Single-Exit
 - For simplicity, C++ includes only single-entry/single-exit control statements—there's only one way to enter and only one way to exit each control statement.
 - Connecting control statements in sequence to form structured programs is simple.
 - ◆ The final state of one control statement is connected to the initial state of the next—that is, the control statements are placed in a program one after another in sequence.
 - ◆ We call this **control-statement stacking**. The rules for forming structured programs also allow for **nested control statements**.

Structured-Programming Summary (Cont.)

- Rules for Forming Structured Programs

- The rules assume that action states may be used to indicate any action. The rules also assume that we begin with the simplest activity diagram consisting of only an initial state, an action state, a final state and transition arrows.

Rules for forming structured programs	
1.	Begin with the simplest activity diagram (Fig. 4.13).
2.	Any action state can be replaced by two action states in sequence.
3.	Any action state can be replaced by any control statement (sequence of action states, if, if...else, switch, while, do...while or for).
4.	Rules 2 and 3 can be applied as often as you like and in any order.

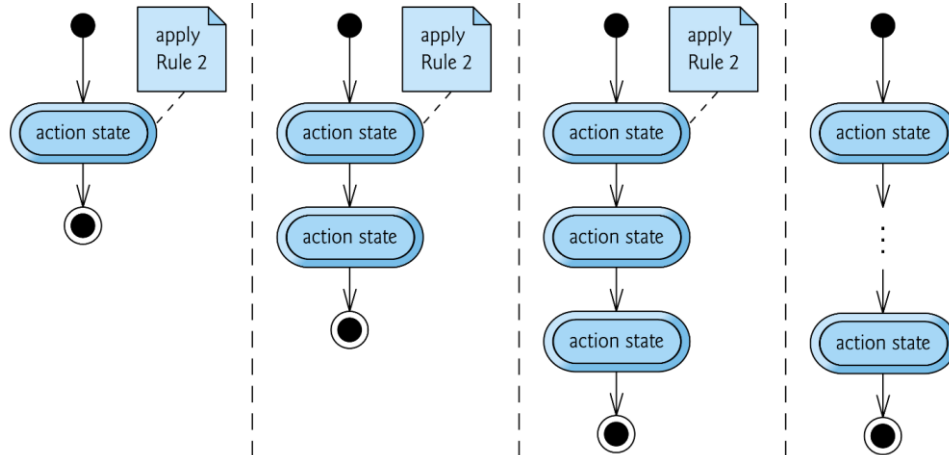


Structured-Programming Summary (Cont.)

- Rules for Forming Structured Programs

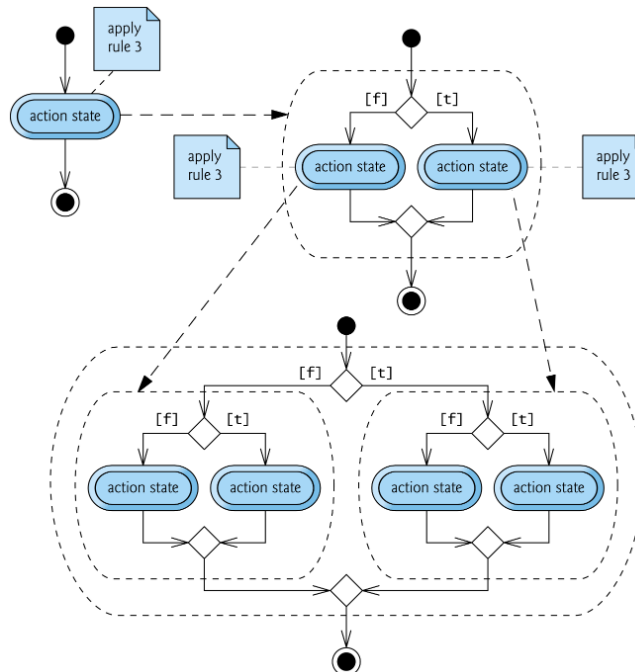
- Applying the rules always results in a properly structured activity diagram with a neat, building-block appearance.

- ◆ For example, repeatedly applying rule 2 (stacking rule) to the simplest activity diagram results in an activity diagram containing many action states in sequence.



Structured-Programming Summary (Cont.)

- Rules for Forming Structured Programs
 - Rule 3 is called the nesting rule. Repeatedly applying rule 3 to the simplest activity diagram results in one with neatly nested control statements.

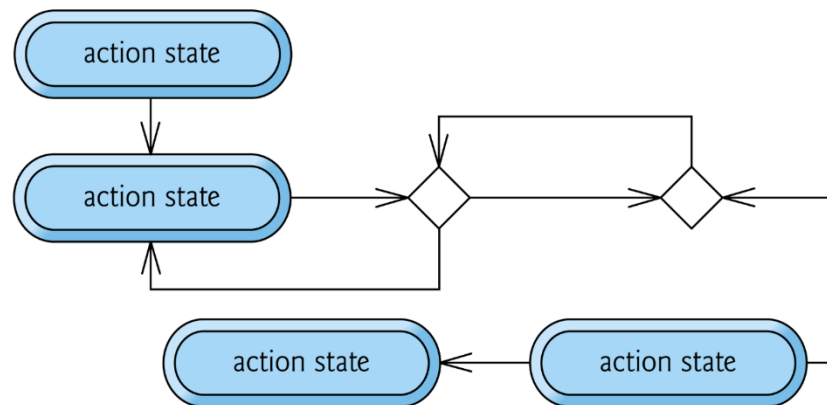


Structured-Programming Summary (Cont.)

- Rules for Forming Structured Programs
 - Rule 4 generates larger, more involved and more deeply nested statements.
 - The beauty of the structured approach is that we use only seven simple single-entry/single-exit control statements and assemble them in only two simple ways.

Structured-Programming Summary (Cont.)

- Rules for Forming Structured Programs.
 - If the rules are followed, an “unstructured” activity diagram cannot be created.
 - ◆ If you’re uncertain about whether a particular diagram is structured, apply the rules in reverse to reduce it to the simplest activity diagram.
 - ◆ If you can reduce it, the original diagram is structured; otherwise, it’s not.



Structured-Programming Summary (Cont.)

- Three Forms of Control

- Structured programming promotes simplicity. Only three forms of control are needed to implement an algorithm:

- ◆ sequence
 - ◆ selection
 - ◆ iteration

Structured-Programming Summary (Cont.)

- Three Forms of Control

- The sequence structure is trivial. Simply list the statements to execute in the order in which they should execute. Selection is implemented in one of three ways:
 - ◆ if statement (single selection)
 - ◆ if...else statement (double selection)
 - ◆ switch statement (multiple selection)
- In fact, it's straightforward to prove that the simple if statement is sufficient to provide any form of selection—everything that can be done with the if...else statement and the switch statement can be implemented by combining if statements (although perhaps not as clearly and efficiently).

Structured-Programming Summary.

- Three Forms of Control

- Iteration is implemented in one of three ways:

- ◆ while statement
 - ◆ do...while statement
 - ◆ for statement

- It's straightforward to prove that the while statement is sufficient to provide any form of iteration. Everything that can be done with do...while and for can be done with the while statement (although perhaps not as conveniently).

Structured-Programming Summary.

- Three Forms of Control

- Combining these results illustrates that any form of control ever needed in a C++ program can be expressed in terms of
 - ◆ sequence
 - ◆ if statement (selection)
 - ◆ while statement (iteration)
- and that these can be combined in only two ways—**stacking** and **nesting**.

Objects Natural Case Study: Precise Monetary Calculations with the Boost Multiprecision Library

- Many applications require precise representations of numbers with decimal points.
 - Institutions like banks that deal with millions or even billions of transactions daily have to tie out their monetary transactions “to the penny.”
- We mentioned earlier that using double values to process monetary amounts, as in compound-interest example, introduces representational error and rounding issues that could prevent “to the penny” accuracy:
 - There were representational errors because we stored precise decimal monetary amounts and interest rates as doubles.
 - There was rounding of the floating-point values because the interest calculations often result in fractional pennies. Those must be rounded to the hundredths place to represent cents. In situations with floating-point representational errors, rounding can be unpredictable.

Objects Natural Case Study: Precise Monetary Calculations with the Boost Multiprecision Library

- Often, there are many points in monetary calculations in which rounding may occur.
 - For example, on a restaurant bill, the tax could be calculated on each individual item, resulting in separate rounding operations, or it could be calculated only once on the total bill amount.
 - These alternate approaches could yield different results.

Objects Natural Case Study: Precise Monetary Calculations with the Boost Multiprecision Library (Cont.)

- Custom Types for Precise Monetary Calculations
 - You'll see in Chapter 9, Custom Classes, that C++ is an extensible language. So, we could create a custom class for monetary calculations and other applications that require precise representation and manipulation of numbers with decimal points.
 - However, getting it right might take significant time and effort.
 - Rather than “reinventing the wheel,” we can take advantage of pre-existing libraries such as the Boost Multiprecision open-source library, which provides types for to-the-penny precision.
 - We'll create Boost Multiprecision objects and use Boost's operators and functions to work with those objects as conveniently as built-in types like double.

Objects Natural Case Study: Precise Monetary Calculations with the Boost Multiprecision Library (Cont.)

- Representing and Calculating with Monetary Amounts Is a Complex Issue
 - Processing monetary amounts can be far more complex than what we show here.
 - ◆ Banks have to deal with issues such as using a “fair rounding algorithm” when calculating daily interest on their accounts.
 - ◆ Rounding rules can vary by currency, government, company, etc.
 - ◆ Also, there are a great many currencies worldwide with different conventions for thousands separators, decimal separators, currency symbols, and more.
 - ◆ Various C++ libraries deal with issues like these.

Objects Natural Case Study: Precise Monetary Calculations with the Boost Multiprecision Library (Cont.)

- Demonstrate Boost Multiprecision `cpp_dec_float_50`
 - To eliminate the representational errors associated with floating-point types, the Boost Multiprecision library's `cpp_dec_float_50` class uses an internal coding scheme that requires additional memory to hold numbers having up to 50 digits of precision.
 - Using this class, we can create objects that represent exact monetary amounts like 123.02 and perform to-the-penny arithmetic without representational error.
 - The next program reimplements the compound interest calculation using `cpp_dec_float_50` objects to represent monetary amounts and interest rates precisely and perform precise arithmetic using them.

Objects Natural Case Study: Precise Monetary Calculations with the Boost Multiprecision Library (Cont.)

- Demonstrate Boost Multiprecision `cpp_dec_float_50`

```
1 // Compound-interest example with boost::multiprecision::cpp_dec_float_50.
2 #include <boost/multiprecision/cpp_dec_float.hpp>
3 #include <format>
4 #include <iostream>
5 #include "decimalformatter.h"
6
7 using namespace std;
8 using boost::multiprecision::cpp_dec_float_50;
9
10 int main() {
11     cpp_dec_float_50 principal{1000}; // $1000 initial principal
12     cpp_dec_float_50 rate{"0.05"}; // 5% interest rate
13
14     cout << format("Initial principal: {:>7}\n", principal)
15           << format("Interest rate: {:>7}\n\n", rate);
16
17     // display headers
18     cout << format("{}{:>20}\n", "Year", "Amount on deposit");
19
20     // calculate amount on deposit for each of 10 years
21     for (int year{1}; year <= 10; ++year) {
22         cpp_dec_float_50 amount{principal * pow(1 + rate, year)};
23         cout << format("{:>4}{:>20}\n", year, amount);
24     }
25 }
```

Initial principal: 1000.00	
Interest rate: 0.05	
Year	Amount on deposit
1	1050.00
2	1102.50
3	1157.62
4	1215.51
5	1276.28
6	1340.10
7	1407.10
8	1477.46
9	1551.33
10	1628.89

Objects Natural Case Study: Precise Monetary Calculations with the Boost Multiprecision Library (Cont.)

- Demonstrate Boost Multiprecision `cpp_dec_float_50`

```
1 // decimalformatter.h
2 // Custom std::formatter that formats a
3 // boost::multiprecision::cpp_dec_float_50 as a std::string
4 // with two digits to the right of the decimal point.
5 #pragma once
6 #include <boost/multiprecision/cpp_dec_float.hpp>
7 #include <format>
8 #include <iomanip>
9 #include <sstream>
10 #include <string>
11
12 // short name for boost::multiprecision::cpp_dec_float_50
13 using decimal = boost::multiprecision::cpp_dec_float_50;
14
15 // Custom formatter for boost::multiprecision::cpp_dec_float_50.
16 // Formats as a string with two digits to the right of the decimal point.
17 // More info at: https://fmt.dev/latest/api.html#format-api
18 template<> struct std::formatter<decimal> : std::formatter<string_view> {
19     template<typename FormatContext>
20     auto format(decimal d, FormatContext& ctx) const {
21         std::ostringstream out{};
22         out << std::fixed << std::setprecision(2) << d;
23         return formatter<string_view>::format(out.str(), ctx);
24     }
25 };
```

Objects Natural Case Study: Precise Monetary Calculations with the Boost Multiprecision Library (Cont.)

- Initializing `cpp_dec_float_50` Objects

- You typically create a `cpp_dec_float_50` object from
 - ◆ an integer value, which C++ can represent precisely, or
 - ◆ a string that specifies the exact digits you wish to represent to the left and right of the decimal point.
- Here, we initialized `principal` with the integer 1000 (line 11) and `rate` with the string "0.05" (line 12).

```
11  cpp_dec_float_50 principal{1000}; // $1000 initial principal
12  cpp_dec_float_50 rate{"0.05"}; // 5% interest rate
```

Objects Natural Case Study: Precise Monetary Calculations with the Boost Multiprecision Library (Cont.)

- C++20 Text Formatting and `cpp_dec_float_50` Objects
 - Boost Multiprecision types do **NOT** yet support C++20 text formatting with placeholders like `{:.2f}` that round a floating-point number to a specified precision.
 - However, these types **do support** the old-style C++ formatting with the fixed and **`setprecision`** stream manipulators.
 - It's also possible to customize C++20 text formatting for custom class types.

Objects Natural Case Study: Precise Monetary Calculations with the Boost Multiprecision Library (Cont.)

- C++20 Text Formatting and `cpp_dec_float_50` Objects
 - So, for this example, we defined in `decimalformatter.h` (line 5) custom formatting that enables `format` to get a string representation of a `cpp_dec_float_50` object rounded to two digits of precision.
 - ◆ Our custom formatter uses the `fixed` and `setprecision` stream manipulators.
 - ◆ You simply `#include` our header to enable `format` to use the custom formatter.

```
1 // Compound-interest example with boost::multiprecision::cpp_dec_float_50.
2 #include <boost/multiprecision/cpp_dec_float.hpp>
3 #include <format>
4 #include <iostream>
5 #include "decimalformatter.h"
```


Objects Natural Case Study: Precise Monetary Calculations with the Boost Multiprecision Library (Cont.)

- C++20 Text Formatting and `cpp_dec_float_50` Objects

- Lines 14–15 use the placeholder `{:>7}` to format principal and rate as strings that are each right aligned (`>`) in a field of 7 character positions.

```
14      cout << format("Initial principal: {:>7}\n", principal)  
15      << format("      Interest rate: {:>7}\n\n", rate);
```

- Line 23 uses the placeholder `{:>20}` to format amount as a string that's right aligned (`>`) in a field of 20 character positions.

```
23      cout << format("{:>4}{:>20}\n", year, amount);
```

- In this case study, we always format `cpp_dec_float_50` monetary amounts with two digits of precision so the right-aligned strings' decimal points align vertically.

Objects Natural Case Study: Precise Monetary Calculations with the Boost Multiprecision Library (Cont.)

- Performing the Interest Calculations

- Line 22's calculation uses `cpp_dec_float_50` objects rather than doubles to avoid type double's representational errors.
- Boost Multiprecision types support the built-in arithmetic operators (such as `*` and `+`) and common math functions (such as `pow`), so those are used here.
 - ◆ They know how to perform precise arithmetic using `cpp_dec_float_50` objects.

```
20 // calculate amount on deposit for each of 10 years
21 for (int year{1}; year <= 10; ++year) {
22     cpp_dec_float_50 amount{principal * pow(1 + rate, year)};
23     cout << format("{:>4}{:>20}\n", year, amount);
24 }
```

Objects Natural Case Study: Precise Monetary Calculations with the Boost Multiprecision Library (Cont.)

- A Policy Issue: Half-Up Rounding vs. Banker's Rounding
 - Compare the outputs of doubles and `cpp_dec_float_50`.
 - ◆ Notice that year 3 has a slightly different output—1157.63 for double vs. 1157.62 for `cpp_dec_float_50`.
 - ◆ This is due to the different rounding algorithms used for doubles and `cpp_dec_float_50`s and could also be due to representational errors when calculating with doubles.
 - In year 3, line 23 calculates $1000.00 * \text{pow}(1 + 0.05, 3)$ and `pow` returns 1.157625, so the preceding calculation yields 1157.625.
 - By default, double amounts use half-up rounding—for two digits to the right of the decimal point, if the third digit is 5 or higher, we round the previous digit up; otherwise, we round it down.

Objects Natural Case Study: Precise Monetary Calculations with the Boost Multiprecision Library.

- Banker's Rounding

- Unfortunately, the commonly used half-up rounding is a **biased** technique.
 - ◆ For the digits 1, 2, 3 and 4, we round the preceding digit down, and for 5, 6, 7, 8 and 9, we round the preceding up—four values round down and five round up.
 - ◆ More values round up than down, leading to possible discrepancies in monetary calculations.

Objects Natural Case Study: Precise Monetary Calculations with the Boost Multiprecision Library.

- Banker's Rounding

- To fix this bias, Boost's `cpp_dec_float_50` uses banker's rounding—when the digit used to determine rounding is 5 (the third digit in this example), this algorithm rounds the previous digit (the second digit in this example) to the **nearest even integer**.
- So with `cpp_dec_float_50`, the value 1157.625 rounds down to 1157.62. Similarly,
 - ◆ 1157.635 would round up to 1157.64,
 - ◆ 1157.645 would round down to 1157.64,
 - ◆ 1157.655 would round up to 1157.66,
 - ◆ 1157.665 would round down to 1157.66, ... etc.